BCSE307P Compiler Design Lab

Innovative Project

AI Based Language Translation Compiler

by

Reg No	Team Member Names
24BCE2952	Sakrit Kafle
24BCE2952	Abendra Adhikari
24BCE2953	Ankur Bhattarai

Under the Supervision of

Prof. MUTHUNAGAI S U

Assistant Professor Sr. Grade-2

School of Computer Science Engineering and Information Systems

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

1. Abstract:

This project is a source to source compiler. It reads a program written in a defined subset of Python and produces a complete C program that can be built with any ordinary C compiler. It is built around a full compiler pipeline of four phases, which are lexical analysis, parsing, semantic analysis and code generation. A desktop development environment is included so that each phase can be inspected while it runs.

The project is described as hybrid because a language model is allowed to take part in three clearly marked places. It is never allowed to decide anything on its own. In every one of the three places, the answer the model gives is checked by something the model did not write, and the answer is thrown away when that check fails. The first place is a gate that handles a call to a function the compiler has never heard of. The second place is an optional pass that tidies the generated C. The third place is a repair pass that rewrites a whole program the compiler refuses to translate.

Correctness is not claimed on the basis of inspection. Every sample program is compiled and then run, and its output is compared with the output of the same program under CPython. The comparison is made on raw bytes rather than on decoded text, because decoding would hide a real difference in line endings. The project currently passes 632 automated tests. The repair pass was measured against a corpus of twelve ordinary Python programs that the compiler refuses, and it produced a verified rewrite for ten of them.

The central idea of the project is that a language model is useful to a compiler only when the compiler can check the answer. Where a check is possible the model is allowed to help. Where a check is not possible the model is not consulted at all.

2. Introduction:

2.1 The problem this project addresses

Python is easy to write and slow to run. C is harder to write and much faster to run. A tool that turns one into the other is therefore useful, and writing one is a direct exercise in every topic of a compiler design course. That is the starting point of this project.

The difficulty is that the obvious translation is wrong. A reader new to the problem would expect Python division to become C division, Python modulo to become C modulo, and a Python print call to become printf. Each of those three mappings produces code that compiles without a single warning and then prints the wrong answer. The table below lists the cases this project had to solve before anything else could be trusted.

Python expression	Python result	What naive C gives
7 / 2	3.5	3, because C integer division truncates
-7 // 2	-4	-3, because C truncates instead of flooring
-7 % 2	1	-1, because C takes the sign of the dividend
2 ** -1	0.5	no direct operator exists in C at all
1 / 0	raises an error	undefined behaviour
print(3.5)	3.5	3.500000 from the default printf format
print(True)	True	1, because C has no boolean word
round(2.5)	2	3, because C rounds halves away from zero

Two further problems are structural rather than arithmetic. In Python a name belongs to the whole function, while in C a name belongs to the block it is declared in. A variable first assigned inside an if statement would therefore disappear at the end of that statement in C. The code generator solves this by lifting every local variable to the top of the C function. The second problem is loop bounds. Python evaluates the argument of range once, while the obvious C for loop would evaluate it on every iteration, so the bounds are computed into temporary variables before the loop starts.

2.2 Why a language model is involved at all

A compiler written by hand knows exactly the rules it was given. When a program calls a function that is not defined in that program and is not one of the builtins the compiler knows, there is nothing sensible for a fixed rule set to do. The usual answer is to stop with an error. That answer is correct but unhelpful, because many of those calls have a short and obvious C equivalent that simply was not written down.

A language model is good at exactly that kind of gap. It can propose that the square root function in Python corresponds to the square root function in the C mathematics library. The problem is that a model can also propose something that is wrong, or something that is dangerous, and it will do so with the same confidence. A model answer that is simply trusted would turn a compiler into a guess.

The design principle of this project follows from that. The model is allowed to propose, and something that the model did not write decides whether the proposal is used. For a single call that decision is made by a structural safety check and then by a real C compiler. For a whole program rewrite that decision is made by the compiler itself and then by running the programs and comparing their output byte by byte. In all three places the failure behaviour is the same. If the check does not pass, the answer is discarded and the user is told why.

2.3 What was built

The deliverable is a working compiler of about 9400 lines of Python source, a grammar file of 245 lines that states exactly which language is accepted, a development environment of about 2700 lines built on the standard tkinter library, and a test suite of 632 tests. The compiler runs with the model switched off by default, and every sample program in the repository compiles and runs correctly in that mode.

3. Novelty in Project:

The novel part of this project is not that a language model was used. Many tools call a model. The novel part is the discipline placed around it. Six specific ideas are set out below.

3.1 The model is bounded by verification rather than by trust

Every one of the three places a model is consulted is paired with a check performed by something else. The table below states what each place is asked and what decides the outcome.

Where	What the model is asked	What decides
Gate, in gate.py	One C expression for one unknown call	A structural safety check, then a real C compiler
Simplify, in simplify.py	Tidier C for a working translation	Both programs are built and run, outputs compared
Repair, in repair.py	A rewrite of a program that will not compile	The whole compiler, then four runs compared byte for byte

A useful consequence is that the compiler never fails quietly. If no proposal passes, compilation stops with an error. There is no fallback that inserts an unchecked stub.

3.2 Safety is checked before compilation, not after

It is tempting to treat a successful compile as proof that a model answer is safe. It is not. A C compiler will happily accept a call that deletes files. For that reason the safety check runs first and is structural. It walks the proposed expression and allows only the arguments of the original call, numeric literals, operators, and a fixed list of pure mathematical functions. Anything else is rejected before a compiler is ever invoked.

3.3 Repair at the level of the source program

The usual response to a program that uses a feature outside the subset is to report an error and stop. This project adds a second option. The model is asked to rewrite the program so that it stays inside the subset and still prints exactly the same output. The rewrite is then passed through the ordinary compiler. Nothing is special cased for it, and the text the model produced never reaches the C compiler directly. Only C that this compiler generated is ever built.

The rewrite does not have to resemble the original program. Only the output is compared, so the model is instructed to reimplement rather than to translate line by line. A decorated function becomes an ordinary function. A list comprehension becomes a loop. An iterator becomes an index.

3.4 Equivalence is established on bytes, from four runs

A claim that two programs behave identically is only as good as the comparison behind it. Three decisions make this comparison strong. First, the original program is run twice, so that a program whose output varies is detected and refused before any model call is made. Second, the rewrite is run under CPython and the generated C is run twice, which gives four runs in total. A mistake by the model and a matching mistake by the compiler cannot cancel out, because neither of them produced the output being compared against. Third, every run is captured as raw bytes. Capturing text instead would translate line endings on both sides of the comparison and would therefore hide a real difference.

3.5 A check that the answer was computed and not copied

If only the output is compared, and the program takes no input, then a program that simply prints the expected answer and computes nothing would pass every test. This was confirmed experimentally during development by writing such a program by hand. It compiled, it ran, and its output matched byte for byte.

The defence rests on a property of the subset, which is that the subset cannot build a string at run time. There is no concatenation, no str function and no formatting. Every piece of text in the output is therefore either a literal or a number, so the only thing a rewrite can copy is a number it should have worked out. A rewrite is rejected before it is run if it prints a run of two or more digits that appears in the expected output and nowhere in the original source, or if it performs no arithmetic and no looping while the original program does both.

3.6 The failure and the resolution are shown side by side

When a translation fails in the development environment, a comparison window opens. The program that did not compile is shown on the left with the failing line marked, and the accepted rewrite is shown on the right. The two are aligned line by line so that an unchanged line appears at the same height on both sides. The evidence is shown underneath, including the compiler error, the stage that rejected each attempt, and the four runs that were compared. A button repeats the whole comparison on demand so that it can be watched rather than believed.

3.7 Coverage is measured rather than asserted

A statement that a tool handles most programs is an opinion unless there is a number behind it. A corpus of twelve ordinary Python programs was therefore built. Each one is refused by the deterministic compiler, and each one was checked to run cleanly and repeatably under CPython, so each one is a fair question to ask. A separate test file keeps the corpus honest without consuming any model credit, by asserting that every program in it still parses, is still refused, and can still serve as a reference.

4. Proposed Work:

4.1 The accepted language

The compiler accepts a defined subset of Python rather than the whole language. The subset is written out in full in the file grammar/subset.ebnf, which is 245 lines long and lists both what is accepted and what is deliberately excluded. Writing the subset down was a deliberate decision, because a subset that exists only in the source code of a compiler cannot be argued about.

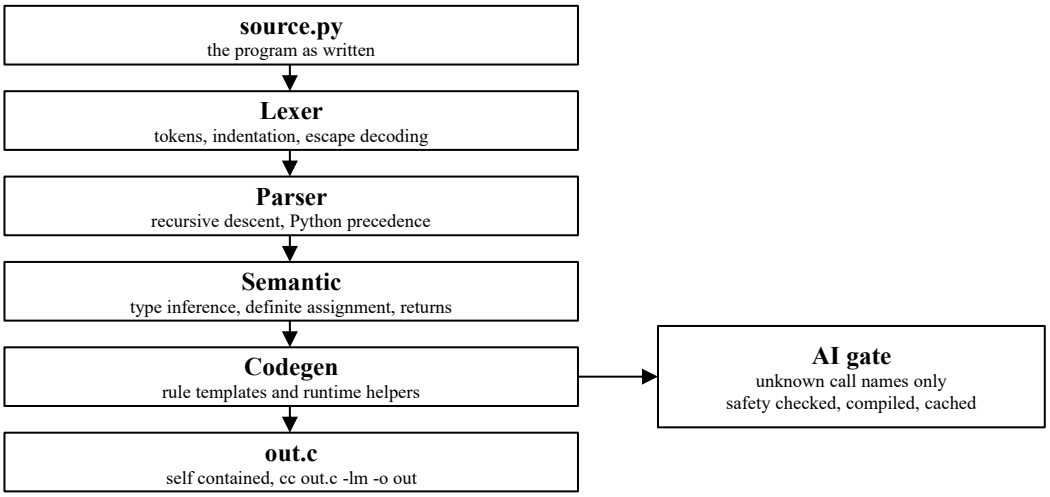
The following features are supported. Integers, floating point numbers, booleans and strings are available as scalar types. Functions with annotated parameters and an explicit return type are supported, including recursion. The usual control flow of if, elif, else, while and for over range is supported. Fixed size lists are supported, where the length is part of the type. Dictionaries with a fixed set of keys are supported. Classes are supported as value structures with fields, methods and constants.

The following features are refused, and each refusal produces a message that names the feature. Growable lists, sets, tuples, comprehensions, slicing, decorators, lambdas, generators, exceptions, inheritance, closures, nested functions, chained assignment and comparison, default and keyword arguments, string indexing and iteration, and string concatenation and formatting are all outside the subset.

The reason for most of these exclusions is the same. The generated C has no memory allocator and no ownership model. A growable list would need one. String concatenation would need one, because a new string has to be stored somewhere. Inheritance with method overriding would need a table of function pointers. The project takes the position that refusing a feature with a clear message is better than half implementing it and producing a program that leaks memory or crashes.

4.2 Architecture

The compiler is a conventional four phase pipeline. The model takes no part in any of the four phases. It is consulted only where the diagram shows it, which is after the semantic phase has already decided that a particular call has no static rule.



4.3 Phase one, lexical analysis

The lexer is written by hand rather than generated. It turns the source text into a stream of tokens. The part that makes Python harder than a brace delimited language is indentation, because indentation is significant and has to become explicit structure. The lexer therefore tracks a stack of indentation levels and emits an indent token when a line is more indented than the previous one and one or more dedent tokens when it is less indented. It also normalises Windows line endings, expands tabs consistently, and decodes escape sequences inside string literals so that later phases never see a backslash that still needs interpreting.

4.4 Phase two, parsing

The parser is a recursive descent parser that builds an abstract syntax tree. Operator precedence follows CPython exactly, which matters because a difference in precedence would silently change the meaning of an expression rather than cause an error. When the parser meets a construct that is outside the subset, it does not produce a generic syntax error. It produces a message that names the construct and explains why it is not supported, which is far more useful to the person holding the program.

4.5 Phase three, semantic analysis

Three separate analyses run over every function in this phase.

- Type inference. This is a small monotone dataflow analysis. Types are allowed to widen along the order boolean, then integer, then floating point, and they are never allowed to narrow. Because the order is finite and movement is only ever in one direction, repeating the analysis until nothing changes is guaranteed to terminate. This is what allows a variable first assigned the integer zero and later assigned the result of a division to be inferred as a floating point variable instead of being rejected.
- Definite assignment. Using a variable before an assignment that is guaranteed to have happened is reported as a compile error. In Python the same mistake produces an error at run time. In C it would read whatever happened to be in that memory, which is far worse, so catching it here is the only acceptable option.
- Return coverage. A function that declares a return type must return a value on every path through it. C would allow the function to fall off the end and return whatever value happened to be in the return register.

This phase is also where a call is classified. If the callee is defined in the program it is an ordinary call. If it is one of the builtins with a static rule it is a builtin call. If it is neither of those, it is marked as needing the gate. That mark is the only route by which a model is ever reached.

4.6 Phase four, code generation

The code generator walks the typed syntax tree and emits C. Each construct has a template. The semantic differences listed in the introduction are handled by a small

runtime that is emitted together with the program. Only the helper functions that the program actually uses are emitted, so a program doing simple arithmetic does not carry code for dictionaries. The result is a single self contained C file that needs no headers or libraries beyond the standard ones and the mathematics library.

4.7 The first place a model is used, the gate

The gate handles a call to a name that the compiler does not know. The question put to the model is deliberately narrow. It is asked for one C expression that is equivalent to the call, written over placeholders named a0, a1 and so on rather than over the actual arguments. Asking for a template rather than an expression for one call site means one verified answer serves every call to that name with that number of arguments.

The answer passes through three steps. It must first pass the structural safety check described earlier. It must then compile with a real C compiler. If no C compiler is installed on the machine, the validation fails rather than passing by default, which is the safe direction to fail in. If either step fails, the failure is described back to the model and it tries again, up to three attempts in total. If no attempt succeeds the compilation fails with a clear message.

An accepted template is written to a cache file. A later build of the same program therefore makes no model call at all and produces exactly the same C. This matters for a compiler, because a tool that produces different output on two identical runs of the same input is not usable. Sampling is also pinned to a temperature of zero for the same reason.

The clearest example of the gate earning its place is the function that converts radians to degrees. There is no C function of that name to rename it to, so the model has to derive the expression that multiplies by one hundred and eighty and divides by pi. The compiler then proves that this derivation compiles before it is used.

4.8 The second place a model is used, simplify

This is an optional pass that asks the model to produce tidier C for a translation that already succeeded. The answer is accepted only after three checks in a fixed order. A safety check runs first, over code with comments and string literals removed, and refuses any access to processes, files, the network or the environment, as well as any header the original did not already use. The safety check runs first precisely because the next steps execute the result. The tidied code must then compile. Finally both programs are run and their output and exit status are compared.

Both outcomes occur in practice. In one recorded case the pass reduced a program from 213 lines of C to 99 lines and was accepted. In another the model dropped a percent sign from a format string. The result compiled cleanly and looked correct. Only running it revealed that one line of output had been corrupted, and the pass was rejected.

4.9 The third place a model is used, repair

Repair is used when the program does not compile at all. Five conditions must hold before a rewrite is accepted, and they are checked in this order.

- The original program must agree with itself. It is run twice before anything else happens. A program that uses the clock or randomness has no fixed output to preserve, so it is refused here, before a single model call is made.
- The rewrite must compile. Not merely look plausible, but pass through the real lexer, parser, semantic analyser and code generator. If it does not, the compiler message is sent back to the model and it tries again.
- The rewrite, run under CPython, must print what the original printed. This is safe to do only at this point, because compiling has proved that the program is inside a subset with no way to express opening a file or reaching the network.
- The generated C must build.
- That binary, run twice, must print what the original printed, both times.

Up to five rewrites are attempted. Each attempt is told everything that every earlier attempt got wrong, not merely the most recent failure. This matters because a rewrite replaces the whole file, so an attempt that is told about only one mistake is free to walk back into an earlier one. An attempt is only spent when the previous one was rejected, so a program that succeeds immediately costs one model call.

4.10 Surveying a program before translating it

A compiler stops at the first thing it cannot handle. That is correct behaviour for a compiler and a poor answer to the question of whether a program can be translated at all, because the answer arrives one error at a time. A survey mode was therefore added. It examines the whole file and groups everything outside the subset into three verdicts. Fixable means the subset can express this and the program needs a change. Not yet means the compiler could support it but does not. Never means no amount of work on either side would help, because the feature needs an allocator, dynamic dispatch, or a library that the generated C cannot reach.

4.11 The development environment

The environment is built with the standard tkinter library only, so it needs no third party packages. It shows the Python source on the left and the generated C on the right. Underneath is a clickable pipeline that shows the result of each phase, so the token stream, the syntax tree, the inferred types and the generated code can each be inspected. A terminal tab runs either the Python or the compiled C. Errors highlight the offending source line. Light and dark themes and a translation history are included.

5. Implementation & Screenshots:

This section shows the system running. Every figure is a capture of the real program rather than a drawing. The command line figures were produced by running the command shown at the top of the figure and photographing whatever came back, and the environment figures are photographs of the real window.

One figure needs a note. The account used for model access ran out of credit while these figures were being captured. Figure 6 and Figure 16 therefore replay a rewrite that the model had produced for the same program a short time earlier. Only the model answer is replayed. The safety check, the compiler, the build, all four runs and the byte comparison shown in those figures were performed live at the moment of capture, and the tool announces the replay in its own output where it happens.

```
$ python -m src.driver tests/programs/arithmetic.py --run
wrote tests/programs/arithmetic.c
call sites: 4   AI-resolved: 0   model queries: 0   (AI-assist disabled)
7 / 2 = 3.5
7 // 2 = 3
7 % 2 = 1
-7 / 2 = -3.5
-7 // 2 = -4
-7 % 2 = 1
7 / -2 = -3.5
7 // -2 = -4
7 % -2 = -1
-7 / -2 = 3.5
-7 // -2 = 3
-7 % -2 = -1
2 ** 10 = 1024
2 ** 0 = 1
-2 ** 2 = -4
2 ** -1 = 0.5
2.0 ** 0.5 = 1.4142135623730951
2 ** 3 ** 2 = 512
7.5 // 2 = 3.0
-7.5 % 2 = 0.5
mixed: 3.5 6.0 2.5
```

Figure 1. A program translated and run with the model switched off. The counters on the third line report that no model query was made. The output below shows the division, modulo and power cases that a naive translation gets wrong.

```
$ type tests\programs\arithmetic.c
/* ---------------------------------------------------------------- *
 *  Generated by the hybrid Python-subset -> C compiler.
 *  Self-contained: build with `cc out.c -lm -o out` (no -I needed).
 * ---------------------------------------------------------------- */
#ifdef __MINGW32__
#  define __USE_MINGW_ANSI_STDIO 1   /* make %lld work with MinGW's printf */
#endif
#ifdef _MSC_VER
#  define _CRT_SECURE_NO_WARNINGS 1
#endif

#define _USE_MATH_DEFINES 1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <limits.h>

#ifndef M_PI
#  define M_PI 3.14159265358979323846
#endif
#ifndef M_E
#  define M_E 2.7182818284590452354
#endif

/* Python's types, mapped onto C. `long long` rather than `int` because
 * Python integers are arbitrary precision; this at least gets 64 bits. */
typedef long long   py_int;
typedef double      py_float;
typedef const char *py_str;
typedef int         py_bool;

#define PY_INT_MIN LLONG_MIN
#define PY_INT_MAX LLONG_MAX

   ... 222 lines omitted ...

}

/* C's entry point. Python's main() is emitted above under another
 * name because int main(void) has a fixed signature. */
int main(void)
{
    return (int)py_user_main();
}
```

Figure 2. The start of the generated C for the same program. The file is self contained and carries only the runtime helpers the program actually uses.

```
$ python -m src.driver tests/programs/ai/01_pythagoras.py --run
tests/programs/ai/01_pythagoras.py: line 6: 'hypot' is not defined in this program and is not one of
 the builtins this compiler translates statically (abs, bool, float, int, len, max, min, pow, put, r
ound). Define it, or enable AI-assist with --ai to have the gate propose and validate a C translatio
n.
```

Figure 3. A program that calls a function the compiler does not know, with the model switched off. The compiler refuses it and names the function rather than failing vaguely.

```
$ python -m src.driver tests/programs/ai/01_pythagoras.py --ai --run
wrote tests/programs/ai/01_pythagoras.c
call sites: 10   AI-resolved: 3   model queries: 0
diagonal of 3 x 4 : 5.0
diagonal of 5 x 12: 13.0
missing side      : 12.0
sqrt(2)           : 1.414214
```

Figure 4. The same program with the gate enabled. Three call sites were resolved by the gate and the program now runs and prints correct results.

```
$ python -m src.driver gui.py --check
gui.py: 1054 things outside the subset

FIXABLE - the subset can express these, the code needs a change  (386)
    list literal                      x33   first at line 661
    code at module level              x16   first at line 11
    parameter 'text'                  x15   first at line 139
    parameter 'parent'                x11   first at line 139
    parameter 'theme'                 x11   first at line 139
    def __init__(...)                 x8    first at line 139
    def apply_theme(...)              x8    first at line 151
    parameter '_'                     x7    first at line 171
    parameter 'report'                x7    first at line 956
    parameter 'source'                x6    first at line 722
    parameter 'command'               x5    first at line 139
    parameter 'message'               x5    first at line 1118
    parameter 'data'                  x5    first at line 2183
    parameter 'font'                  x4    first at line 139
    parameter 'kind'                  x4    first at line 139
    parameter 'event'                 x4    first at line 350
    parameter 'tag'                   x4    first at line 927
    parameter 'repaired'              x4    first at line 956
    parameter 'window'                x4    first at line 1810
    parameter 'first'                 x3    first at line 302
    parameter 'last'                  x3    first at line 302
    parameter 'ok'                    x3    first at line 948
    parameter 'stage'                 x3    first at line 948

   ... 269 lines omitted ...

    import src.export                 x1    first at line 1909
    import src.diagnose               x1    first at line 2106
    .upper()                          x1    first at line 2174
    import re                         x1    first at line 2686

This program cannot be translated: 556 of the 1054 are in the NEVER group.
```

Figure 5. The survey mode run on the source of the development environment itself. It reports every construct outside the subset in one pass and sorts them into the three verdicts, instead of stopping at the first one.

```
$ python -m tools.replay_repair demos/repair_me.py --repair --run
demos/repair_me.py: line 15: expected RPAREN, got STRING 'sum of squares 1..10 = {total}'

attempting an AI repair of the source

  .. reference: running the original under CPython to find out what it prints...
  ok reference: the original prints 7 line(s) / 115 bytes and exits 0, twice over - that is the targ
et
  .. model: attempt 1: asking anthropic/claude-haiku-5.5 for a rewrite...

[replaying a previously obtained rewrite: repair_me.repaired.py]
[the account is out of credit; every check below still runs for real]

  .. compile: attempt 1: lexer -> parser -> semantic -> codegen
  ok compile: it compiles - 241 lines of C
  .. verify: attempt 1: running the rewrite under CPython and comparing it with your program
  ok verify: the rewrite prints the same thing under CPython
  .. build: attempt 1: building the C and running it twice
  ok verify: verified byte for byte: 115 bytes of output and exit status 0, the same from your progr
am under CPython, from the rewrite under CPython, and from the compiled C on both of its runs

--- what changed ---
--- demos/repair_me.py
+++ demos/repair_me.repaired.py
@@ -1,23 +1,86 @@
-# A program that is ordinary Python and outside the subset in four ways.
-#
-# It is here to exercise --repair: an f-string, a list comprehension,
-# enumerate(), and string concatenation are each a construct the deterministic
-# core refuses, and none of them changes what the program prints - so a rewrite
-# into the subset has something exact to be checked against.

   ... 98 lines omitted ...

--- what was compared ---
target: 115 bytes, exit 0, sha256 3b5c6b6c292f807d...
  your program under CPython, run twice  identical
  the rewrite under CPython              identical
  the compiled C, run 1                  identical
  the compiled C, run 2                  identical

repair accepted: verified byte for byte: 115 bytes of output and exit status 0, the same from your p
rogram under CPython, from the rewrite under CPython, and from the compiled C on both of its runs
wrote demos/repair_me.repaired.py
wrote demos/repair_me.c
call sites: 27   AI-resolved: 0   model queries: 0   (AI-assist disabled)
sum of squares 1..10 = 385
  0 -> 0
  1 -> 2
  2 -> 4
   ... output continues ...
```

Figure 6. A complete repair run. The compiler refuses the program at line 15, the original is run twice to establish what it prints, the rewrite is compiled and checked, and the table near the bottom records the four runs that were compared. The bracketed note is the replay described above.

```
$ python -m src.driver report_submission/screenshots/_varies.py --repair
report_submission/screenshots/_varies.py: line 6: expected RPAREN, got STRING 'started at {stamp}'

attempting an AI repair of the source

  .. reference: running the original under CPython to find out what it prints...
  no reference: the original program does not print the same thing twice: line 1 is 'started at 107'
, expected 'started at 997'
Nothing can be verified against it, because matching one of its runs would say nothing about the nex
t. A program that uses the clock, randomness, or anything else that varies has no fixed output to pr
eserve.

no repair accepted: the original program does not print the same thing twice: line 1 is 'started at
107', expected 'started at 997'
Nothing can be verified against it, because matching one of its runs would say nothing about the nex
t. A program that uses the clock, randomness, or anything else that varies has no fixed output to pr
eserve.
```

Figure 7. A repair that is correctly refused. The program reads the clock, so it does not print the same thing twice and there is nothing stable to verify a rewrite against. This is decided by running the original twice, before any model call is made.

```
$ python -m tools.repair_bench
repair bench: 12 program(s), up to 3 attempt(s) each

program      verdict  tries  seconds stages (* = failed)
bignum       PASS     1      6.6     verified
cipher       FAIL     3      14.0    compile* > compile* > compile*
counting     PASS     3      20.4    compile* > equivalence* > verified
generators   PASS     1      5.1     verified
growing      PASS     1      5.6     verified
inherit      PASS     2      8.1     compile* > verified
mapping      PASS     2      8.5     compile* > verified
matrix       FAIL     3      14.1    compile* > compile* > compile*
money        PASS     1      6.6     verified
omnibus      PASS     1      8.5     verified
raising      PASS     1      3.5     verified
strings      PASS     1      3.8     verified

10/12 landed
```

Figure 8. The measured result of the repair pass over the twelve program corpus. Ten of the twelve produced a verified rewrite. The last column records which stage rejected each attempt.

```
$ python -m pytest tests/ -q
.................................................................... [ 11%]
.................................................................... [ 22%]
.................................................................... [ 33%]
.....................................sss............................ [ 44%]
.............................................sssssssss.............. [ 55%]
.................................................................... [ 66%]
.................................................................... [ 78%]
.................................................................... [ 89%]
............................................................         [100%]
632 passed, 13 skipped in 74.91s (0:01:14)
```

Figure 9. The automated test suite. The tests cover the lexer, parser, semantic analysis, code generation, the runtime semantics compared against CPython, the model usage policy, and the repair pass.

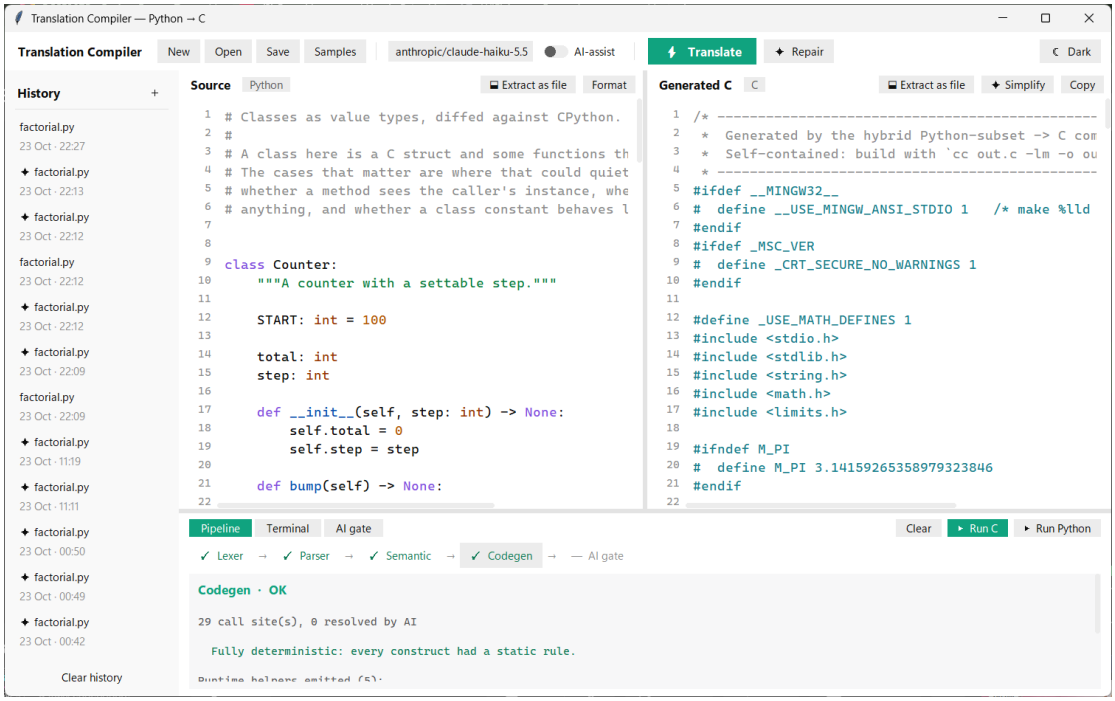


Figure 10. The development environment after a successful translation. Python is on the left, generated C is on the right, and the pipeline across the middle shows that all four phases passed and that the gate was not needed.



Figure 11. The lexical analysis phase. The token count and the individual tokens are listed with the line each came from.

✓ Lexer → ✓ Parser → ✓ Semantic → ✓ Codegen → — AI gate

Parser · OK

```
2 function(s)

{
  "_type": "Program",
  "functions": [
    {
      "_type": "FunctionDef",
      "name": "drain",
      "params": [
        {
          "_type": "Param",
          "name": "g",
          "type": "obj:Grid"
        }
      ],
      "return_type": "int",
      "body": [
        {
          "_type": "Assign",
          "name": "taken",
          "value": {
            "_type": "MethodCall",
```

Figure 12. The parsing phase, showing the abstract syntax tree produced from the token stream.

✓ Lexer → ✓ Parser → ✓ Semantic → ✓ Codegen → — AI gate

Semantic · OK

```
15 function(s), 29 call site(s)

  def Counter____init__(self: obj:Counter, step: int) -> None

  def Counter__bump(self: obj:Counter) -> None

  def Counter__add(self: obj:Counter, n: int) -> None

  def Counter__value(self: obj:Counter) -> int

  def Counter__from_start(self: obj:Counter) -> int

  def Grid____init__(self: obj:Grid) -> None

  def Grid__put(self: obj:Grid, i: int, v: int) -> None

  def Grid__get(self: obj:Grid, i: int) -> int

  def Grid__total(self: obj:Grid) -> int
       s: int
       i: int
```

Figure 13. The semantic analysis phase, showing the types that were inferred for each function and variable.

✓ Lexer → ✓ Parser → ✓ Semantic → ✓ Codegen → — AI gate

Codegen · OK

```
29 call site(s), 0 resolved by AI

  Fully deterministic: every construct had a static rule.

Runtime helpers emitted (5):
  py_index
  py_print
  py_print_bool
  py_print_int
  py_print_str
```

Figure 14. The code generation phase. It reports how many call sites were found and how many were resolved by the gate, which is zero here.

```
$ gcc.exe prog.c -lm -o prog && ./prog
compiled ✓
2 10
5 10
105 110 100
5 1 10
5 7 12 2
18 6 1
18 0
9 0
True False 50 savings
[exit 0]
```

Figure 15. The compiled C program built and run from inside the environment.

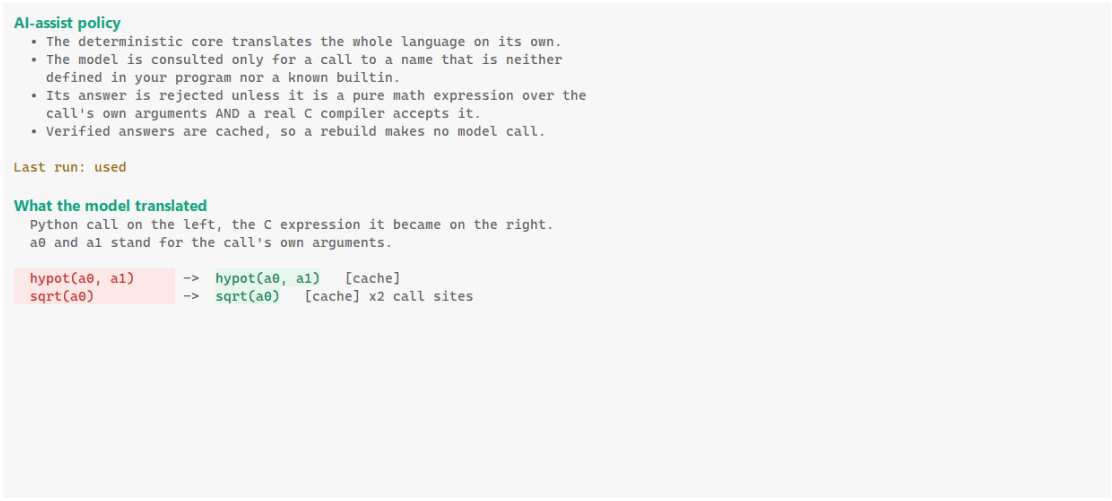


Figure 16. The gate report for a translation that needed it. Each resolved call is listed with the C expression that was accepted and the evidence that it passed the safety check and compiled.

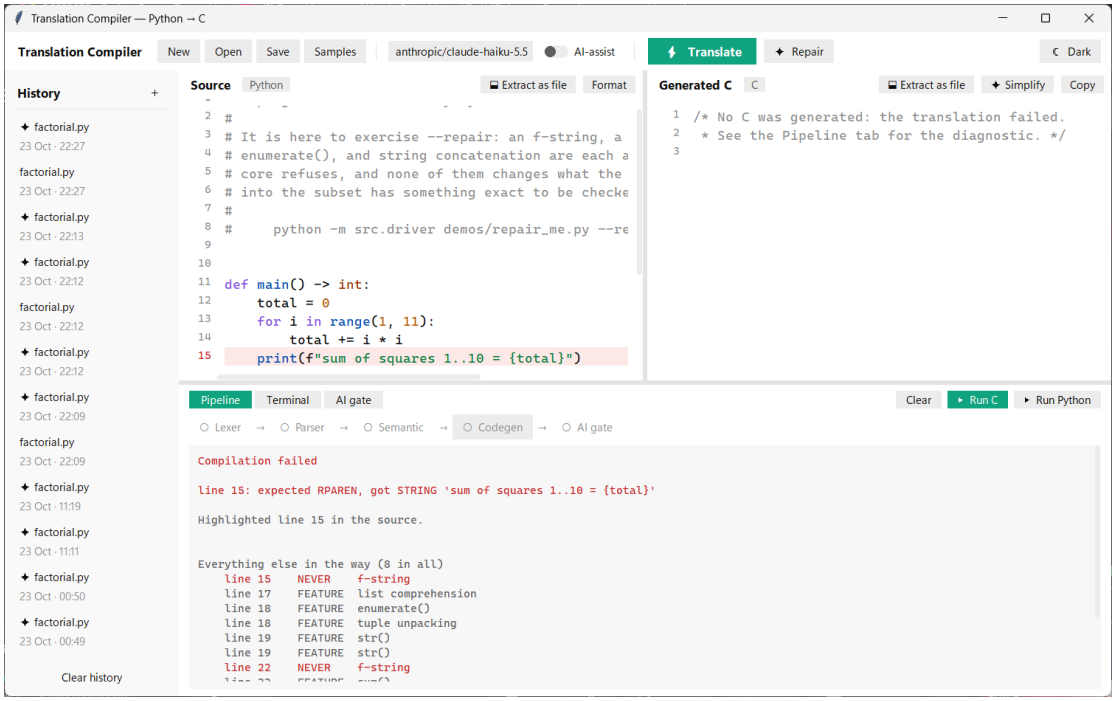


Figure 17. A refused translation with the model switched off. The failing line is highlighted in the source and the whole file survey is shown underneath, so the full picture is visible instead of one error.

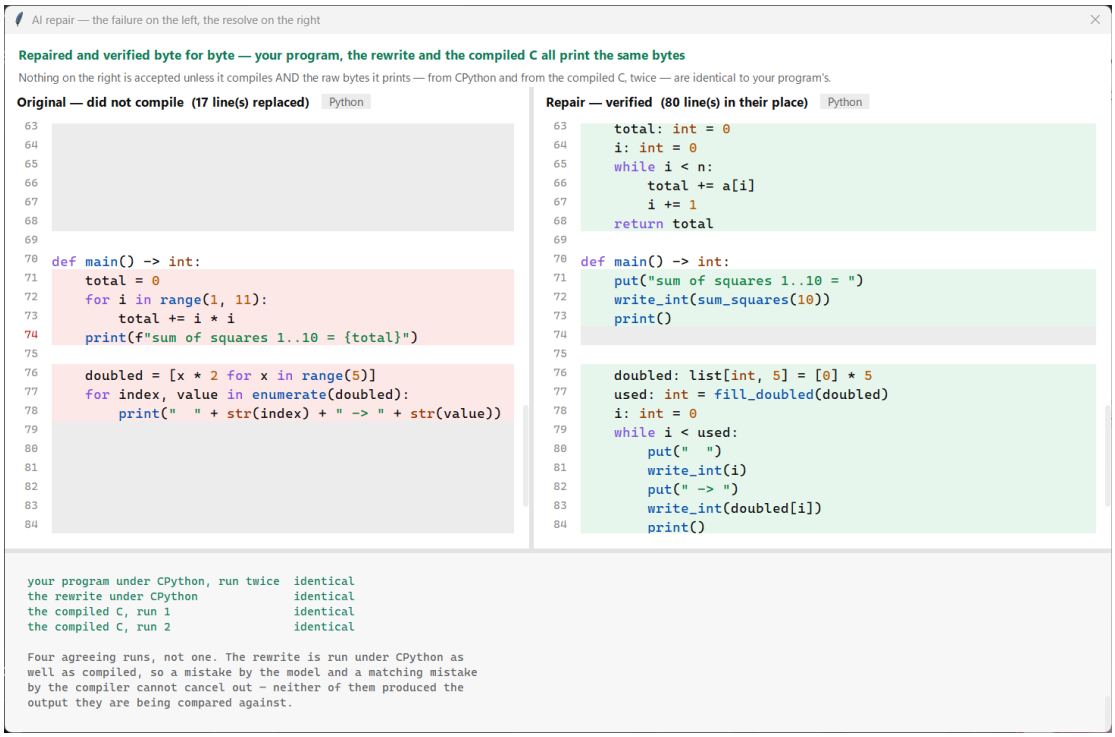


Figure 18. The comparison window. The program that did not compile is on the left and the verified rewrite is on the right, aligned line by line. The four runs that were compared are listed underneath.



Figure 19. The same window after the compare button was pressed. Both programs were run again at that moment and their raw bytes were compared, so the claim can be watched rather than taken on trust.

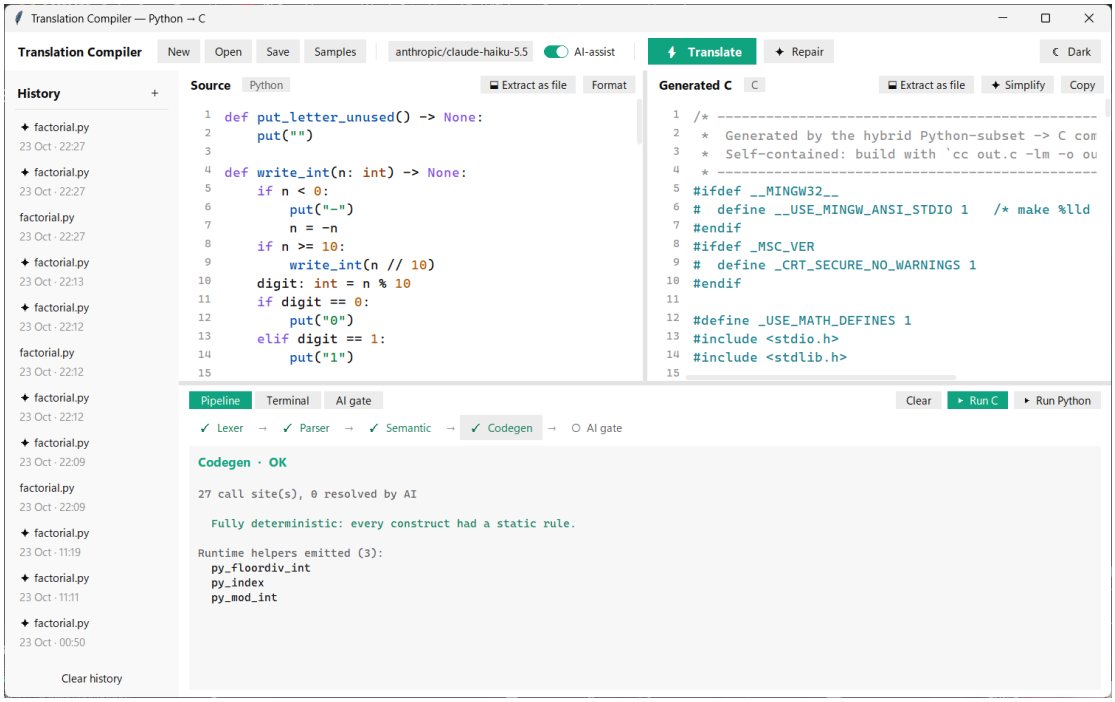


Figure 20. The accepted rewrite moved into the source pane and translated again as an ordinary program. The rewrite is never applied without this step.

6. Conclusion:

6.1 What the project achieves

The project delivers a working compiler for a defined subset of Python that produces correct and self contained C. The four classical phases are implemented by hand, and the semantic differences between the two languages are handled rather than ignored. The claim of correctness rests on 632 automated tests, including tests that build each sample program, run it, and compare its output byte by byte with the output of CPython.

It also demonstrates a usable discipline for involving a language model in a tool that must be correct. The model is consulted in three places, it is given a narrow question in each, and in every case something it did not write decides whether its answer is used. The measured result of the repair pass is that ten of twelve refused programs received a rewrite that was proved equivalent on raw bytes.

6.2 Limitations

The limitations are set out plainly below, with the reason for each. They fall into three groups, which are limits of the language accepted, limits of the generated code, and limits of the model assisted parts.

Limits of the accepted language

- Growable lists are not supported. A list has its length as part of its type. The reason is that growing a list requires a memory allocator and a rule for who frees the memory, and the generated C deliberately has neither.
- Strings cannot be built at run time. There is no concatenation, no str function and no formatting. The reason is the same allocator problem, because a newly built string has to be stored somewhere. Output is composed instead by writing the pieces one after another.
- Strings cannot be indexed or iterated, and there is no function to convert a character to a number or back. This follows from the previous point, and it means that a program which manipulates text character by character has to carry the text as numbers instead.
- Classes have no inheritance and no method overriding. A class is a structure with functions attached. Overriding would require a table of function pointers, and a function is not a value in this subset.
- Dictionaries have a fixed set of keys that is decided when the program is compiled. Keys cannot be added or removed, because the key set is part of the type.
- Exceptions are not supported, so an error has to travel back to the caller as an ordinary return value.
- Arrays hold numbers only, so there is no array of arrays and no array of strings. A two dimensional grid has to be written as one flat array with the index worked out by hand.

- A function cannot return an array. The destination array is passed in as a parameter instead, and the function returns how many entries it filled. This is a direct consequence of how arrays behave in C.

Limits of the generated code

- Integers are 64 bits wide and wrap around on overflow, while Python integers grow without limit. A program whose values exceed 64 bits will compile, will run, and will print a wrong number without any warning. This is the most serious limitation in the project, because unlike every other limitation it produces no error message. It is documented in the grammar file rather than hidden.
- Recursion depth is limited by the C stack rather than by the Python recursion limit, so a deeply recursive program may behave differently from the original.
- Declaring a variable as a floating point number and giving it a whole number value produces a known difference in printed output, because the compiler prints it as a floating point value and CPython prints it as an integer. This case is known and has not yet been resolved.
- Only two standard library modules are recognised. Any other import is refused by name.

Limits of the model assisted parts

- A program that does not finish on its own cannot be repaired. There is no finite output to preserve, so there is nothing a rewrite could be checked against. This is a fact about the program and not a weakness of the model.
- A program whose output varies between runs cannot be repaired, for the same reason. Matching one run of such a program would say nothing about the next.
- A program that prints nothing can be repaired, but the result is reported as unproven rather than verified, because matching an empty output is not evidence of anything.
- The verification applies to the program as it was written and to the input it was given, which for this subset is no input at all. It is a proof about that program, not a proof that the compiler is correct in general.
- Repair only runs when the compiler reports an error. A program that compiles successfully but produces a wrong number, such as the 64 bit overflow case above, never reaches the repair pass at all, because nothing reported a failure.
- The repair pass depends on an external service, so it needs a network connection and an account with credit. The deterministic compiler does not, and it remains fully usable when the service is unavailable.
- Two of the twelve corpus programs are not yet handled. Both compile and both come close to the correct output. They are recorded as open rather than removed from the corpus.

6.3 Future work

The clearest next step is to remove the silent failure described above. If the compiler refused a value that does not fit in 64 bits instead of wrapping, the wrong answer would

become an error message, and an error message is something the repair pass can act on. The second step is to extend the subset to cover the features currently marked as not yet supported, which include raising errors, iterating the entries of a dictionary, and tuple unpacking. The third step is to grow the measured corpus, since every program added to it has so far exposed something worth fixing.

7. **Project Repository Link: (GitHub or drive link)**

   https://github.com/sakrit204-byte/aibased_language_translation_compiler